Universidad del Valle
de Guatemala

Facultad de Ingeniería
Ciencia de la Computación y Tecnologías de la Información
Diseño de Lenguajes de Programación - Sección 10
Catedrático: Carlos Valdez

Proyecto 01

Generador de Analizadores Léxicos

Plataforma Integrada basada en YALex

Diego Lopez #23747
Jennifer Toxcon #21276
Roberto Barreda #23354

Guatemala, 20 de marzo de 2026

Contents

1 Introducción **3**

2 Objetivos **3**
2.1 Objetivo General . . . 3
2.2 Objetivos Específicos . . . 3

3 Alcance del Sistema **4**

4 Marco Teórico **4**
4.1 Expresiones Regulares . . . 4
4.2 Autómatas Finitos Deterministas . . . 5
4.3 Construcción Directa de AFD . . . 5
4.4 Minimización de AFD . . . 5
4.5 Especificaciones YALex . . . 5

5 Arquitectura del Sistema **6**
5.1 Descripción General . . . 6

6 Diseño del Backend **6**
6.1 Parser y Normalizador YALex . . . 7
6.2 Construcción y Minimización de AFD . . . 7
6.3 Runtime y Generación de Código . . . 7

7 Diseño del Frontend **8**

8 Integración entre Frontend y Backend **10**

9 Módulos Funcionales **10**
9.1 Dashboard . . . 10
9.2 Lexical Analysis . . . 10
9.3 Lexical Generator . . . 11
9.4 Test Case Manager . . . 11

10 Generación de Lexer Independiente **11**

11 Casos de Prueba **11**
11.1 Complejidad Baja . . . 12
11.2 Complejidad Media . . . 12
11.3 Complejidad Alta . . . 12

12 Estrategia de Pruebas y Validación **12**

13 Resultados del Sistema **13**

14 Restricciones y Decisiones de Diseño **13**
14.1 Prohibición de Librerías de Regex 13
14.2 Independencia del Lexer Generado 14
14.3 Separación por Responsabilidades 14
14.4 Interfaz Unificada 14

15 Conclusiones **14**

16 Referencias **14**

1. Introducción

El presente informe documenta el diseño y la implementación de una plataforma integrada para la generación, validación y visualización de analizadores léxicos a partir de especificaciones YALex. El sistema combina un motor de compilación léxica en Python, una API de servicios para integración y una interfaz web unificada orientada a la exploración visual del proceso completo de análisis léxico.

Un analizador léxico es la primera fase de un compilador o intérprete. Su responsabilidad consiste en transformar una secuencia de caracteres en una secuencia de tokens con estructura y significado sintáctico. Automatizar la construcción de analizadores léxicos a partir de reglas y expresiones regulares es un problema clásico que integra teoría de lenguajes formales, autómatas finitos, construcción directa de AFD y generación de código.

La plataforma desarrollada cubre el ciclo completo del problema: especificar reglas léxicas en formato `.yal`, parsear y normalizar dichas especificaciones, construir AFDs directos minimizados, generar un lexer Python autosuficiente, analizar entradas reales, visualizar autómatas y ejecutar lotes de casos de prueba desde una sola interfaz.

2. Objetivos

2.1 Objetivo General

Diseñar e implementar una plataforma integrada para la generación de analizadores léxicos basada en YALex, con backend en Python, API de servicios y frontend web moderno, validada con casos de complejidad baja, media y alta.

2.2 Objetivos Específicos

- Implementar la construcción directa de AFD a partir de expresiones regulares mediante el método del árbol sintáctico aumentado.
- Implementar la minimización de AFD mediante refinamiento de particiones.
- Diseñar un parser para especificaciones YALex que soporte definiciones `let`, reglas, argumentos y bloques auxiliares.
- Generar archivos Python autosuficientes que funcionen como analizadores léxicos independientes del generador.
- Exponer las capacidades del motor léxico mediante una API tipada y consistente.
- Construir una interfaz web amigable y estética para compilar especificaciones, analizar entradas, visualizar autómatas y ejecutar casos de prueba.

- Validar el correcto funcionamiento del sistema con casos representativos de complejidad baja, media y alta.

3. Alcance del Sistema

La plataforma se organiza en cuatro módulos funcionales principales:

- **Dashboard**: resume el estado general del sistema y sus métricas operativas.
- **Lexical Analysis**: permite seleccionar una complejidad, cargar o editar una entrada y obtener tokens, errores léxicos y estadísticas de análisis.
- **Lexical Generator**: permite cargar o editar una especificación YALex, compilarla, inspeccionar una regla activa, visualizar su autómata y descargar el lexer generado.
- **Test Case Manager**: permite listar casos de prueba, ejecutar casos individuales o lotes completos y mostrar resultados agregados de validación.

El sistema se concibe como una herramienta integral de experimentación, verificación y documentación del proceso de análisis léxico.

4. Marco Teórico

4.1 Expresiones Regulares

Las expresiones regulares constituyen un formalismo para describir conjuntos de cadenas, es decir, lenguajes regulares. En el contexto de la generación de analizadores léxicos, cada token se define por medio de una expresión regular que describe los lexemas aceptados por la regla correspondiente.

Los operadores fundamentales considerados en el sistema son:

- **Unión** (`|`): representa alternancia entre expresiones.
- **Concatenación**: representa la yuxtaposición de subexpresiones.
- **Cerradura de Kleene** (`*`): cero o más repeticiones.
- **Cerradura positiva** (`+`): una o más repeticiones.
- **Opcionalidad** (`?`): cero o una ocurrencia.

En YALex se soportan, además, conjuntos de caracteres, rangos, conjuntos negados, literales simples y dobles, diferencia de conjuntos (`#`), el comodín `_` y el símbolo especial `eof`. La precedencia adoptada es, de mayor a menor: diferencia, operadores unarios, concatenación y unión.

4.2 Autómatas Finitos Deterministas

Un AFD se define formalmente como una 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$, donde Q es un conjunto finito de estados, Σ es el alfabeto, δ es la función de transición, q_0 es el estado inicial y F es el conjunto de estados de aceptación.

La relevancia de los AFD en este proyecto radica en que permiten ejecutar el reconocimiento léxico de forma determinista, eficiente y completamente independiente de librerías externas de expresiones regulares.

4.3 Construcción Directa de AFD

El método directo construye el AFD a partir del árbol sintáctico de la expresión regular aumentada $r\#$, sin pasar por un AFN intermedio. Para cada nodo del árbol se calculan cuatro funciones:

1. **anulable**(n): indica si el subárbol puede generar la cadena vacía.
2. **primeraPos**(n): conjunto de posiciones que pueden aparecer primero.
3. **últimaPos**(n): conjunto de posiciones que pueden aparecer al final.
4. **siguientePos**(p): posiciones que pueden seguir inmediatamente a una posición dada.

Cada estado del AFD se modela como un conjunto de posiciones del árbol. El estado inicial corresponde a `primeraPos` de la raíz, mientras que las transiciones se construyen a partir de la relación `siguientePos`.

4.4 Minimización de AFD

Una vez construido el AFD, se aplica un algoritmo de minimización por particiones. Dos estados son equivalentes si, para todo símbolo del alfabeto, transitan hacia estados pertenecientes a las mismas clases de equivalencia. El refinamiento iterativo de las particiones produce un AFD mínimo equivalente, reduciendo complejidad estructural y mejorando la claridad de la visualización resultante.

4.5 Especificaciones YALex

YALex (*Yet Another Lex*) define un lenguaje de especificación para analizadores léxicos con la siguiente estructura general:

```
{ header }
let ident = regexp
...
rule entrypoint [arg1, ..., argn] =
    regexp { action }
  | regexp { action }
  | ...
{ trailer }
```

Estructura general de una especificación YALex

Las definiciones `let` permiten factorizar expresiones, mientras que la sección `rule` define patrones y acciones. La semántica de matching utilizada por el sistema es *longest match*, con desempate por orden de declaración.

5. Arquitectura del Sistema

La plataforma se diseña como un sistema por capas, con separación clara entre interfaz, servicios, compilación y ejecución. La Figura 1 resume el flujo principal del sistema.

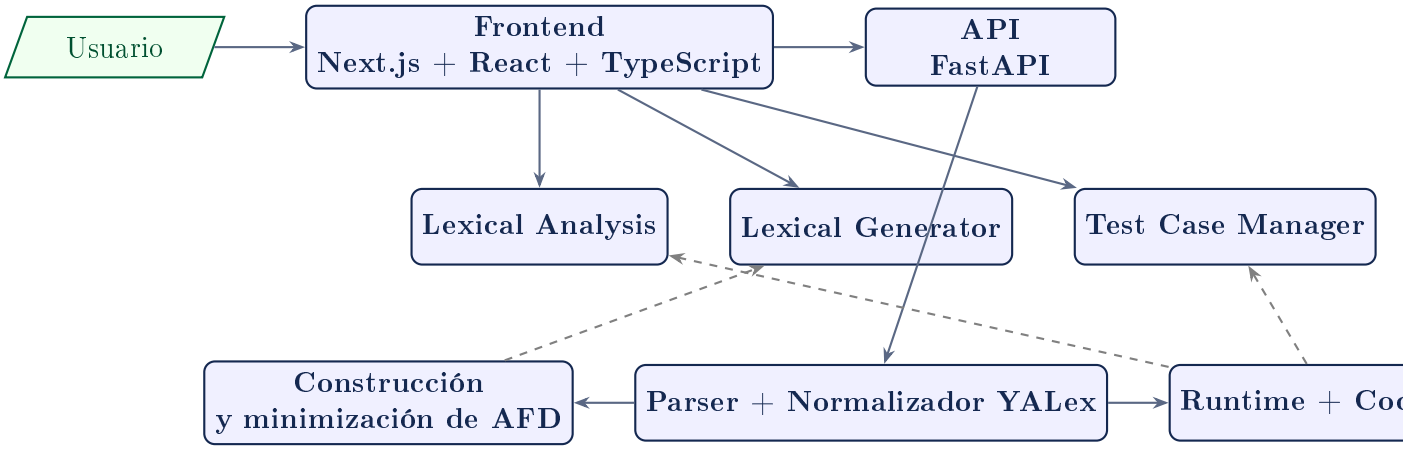

Figure 1: Arquitectura general de la plataforma integrada.

5.1 Descripción General

El frontend web presenta una experiencia integrada que centraliza las operaciones principales del sistema. La API expone contratos estables para los módulos visuales, mientras que el backend conserva la lógica formal del generador y del motor de escaneo. Esta separación permite mantener la exactitud del núcleo léxico en Python y, al mismo tiempo, ofrecer una interfaz moderna para exploración y validación.

6. Diseño del Backend

El backend está implementado en Python y se organiza en componentes especializados por responsabilidad. La lógica de dominio se mantiene separada de la capa de transporte HTTP, lo que facilita pruebas, reutilización y mantenibilidad.

Módulo	Responsabilidad
`analizador.py`	Tokenización de expresiones regulares, inserción de concatenación explícita y conversión a notación postfija.
`arbol.py`	Construcción del árbol sintáctico aumentado y cálculo de *anulable*, *primeraPos*, *últimaPos* y *siguientePos*.
`construccion_afd.py`	Construcción directa del AFD a partir del árbol sintáctico.
`minimizacion.py`	Refinamiento de particiones y generación del AFD mínimo equivalente.
`simulador.py`	Simulación de autómatas sobre cadenas de entrada.
`yalex_parser.py`	Parsing estructural de especificaciones `.yal`.
`yalex_regex.py`	Normalización de sintaxis YALex a la representación interna del motor.
`yalex_compiler.py`	Orquestación de parsing, normalización, construcción de AFD y minimización.
`yalex_runtime.py`	Ejecución del escaneo con semántica de *longest match*.
`yalex_codegen.py`	Generación de un lexer Python autosuficiente.
`api/`	Exposición de endpoints HTTP para el frontend y clientes externos.

6.1 Parser y Normalizador YALex

El parser YALex procesa comentarios, bloques `header` y `trailer`, definiciones `let`, entrypoints con argumentos y reglas separadas por `|`. A continuación, el normalizador convierte la sintaxis de alto nivel a una representación interna uniforme, respetando la precedencia de operadores y resolviendo referencias a definiciones previas.

6.2 Construcción y Minimización de AFD

Cada regla léxica se traduce a un árbol sintáctico aumentado, a partir del cual se obtiene un AFD directo. El autómata resultante se minimiza antes de ser serializado para el runtime o para la visualización del frontend.

6.3 Runtime y Generación de Código

El runtime implementa la política de *longest match*: para cada posición de la entrada, evalúa todas las reglas disponibles, selecciona el prefijo más largo y, en caso de empate, prioriza la regla de menor índice. Las acciones asociadas se ejecutan de acuerdo con la especificación.

El generador de código produce un archivo Python que contiene:

- los AFDs serializados como estructuras nativas;

- las clases y funciones necesarias para el escaneo;
- el entrypoint configurado por la especificación;
- las secciones auxiliares copiadas desde el archivo fuente.

7. Diseño del Frontend

El frontend se implementa dentro de la carpeta `frontend/` como una aplicación de **Next.js**, **React** y **TypeScript**. Su responsabilidad consiste en concentrar la visualización y operación de los módulos del sistema, mientras que el procesamiento formal, la compilación YALex y la ejecución del lexer se mantienen en el backend Python.

La base del proyecto frontend se define a partir de los siguientes archivos:

- `frontend/package.json`: administra dependencias y scripts de desarrollo, pruebas y construcción.
- `frontend/next.config.ts`: configura Next.js y centraliza la reescritura de rutas relativas `/api/*` hacia el backend.
- `frontend/src/app/layout.tsx`: declara el layout raíz compartido por toda la aplicación.
- `frontend/src/app/page.tsx`: monta la vista principal y delega la interfaz al shell de navegación.
- `frontend/src/app/globals.css`: contiene estilos globales, definiciones visuales y reglas compartidas por todos los módulos.

La estructura funcional del frontend se organiza alrededor de un shell principal. Esta capa se implementa en:

- `frontend/src/features/shell/app-shell.tsx`: compone la interfaz general del sistema.
- `frontend/src/features/shell/module-card.tsx`: representa los accesos a cada módulo desde la vista principal.
- `frontend/src/features/shell/module-panel.tsx`: decide qué módulo se renderiza en el panel activo.
- `frontend/src/hooks/use-active-view.ts`: mantiene el estado de la vista seleccionada.
- `frontend/src/lib/modules.ts`: define la metadata de los módulos visibles dentro de la interfaz.

La elección de esta estructura responde a una consideración técnica concreta: la interfaz debía funcionar como una sola aplicación integrada. Por ello, el cambio entre *Dashboard*,

Lexical Analysis, *Lexical Generator* y *Test Case Manager* se resuelve dentro del mismo shell, sin delegarlo a documentos incrustados o a vistas aisladas.

Cada herramienta se implementa como una feature independiente dentro de `frontend/src/features/`:

- `analysis/`: contiene `analysis-tool.tsx` y `use-analysis-tool.ts`, encargados de la entrada de texto, carga de archivos, ejecución del análisis y presentación de tokens y errores.
- `generator/`: contiene `generator-tool.tsx`, `use-generator-tool.ts`, `generator-graph.tsx` y `generator-templates.ts`; estos archivos resuelven la edición de especificaciones YALex, la compilación, la selección de reglas y la visualización del autómata en SVG.
- `test-manager/`: contiene `test-manager.tsx` y `use-test-manager.ts`, responsables de cargar el catálogo de pruebas, ejecutar casos individuales o masivos y calcular métricas de sesión.
- `shell/`: agrupa los componentes estructurales que mantienen la navegación y el ensamblaje general de la interfaz.

El intercambio de datos y artefactos se concentra en archivos auxiliares compartidos:

- `frontend/src/lib/api.ts`: implementa la capa de acceso HTTP y la lectura de respuestas JSON.
- `frontend/src/lib/download.ts`: gestiona la descarga de tokens, del lexer generado y de representaciones serializadas del autómata.
- `frontend/src/types/analysis.ts`, `frontend/src/types/generator.ts`, `frontend/src/types/` y `frontend/src/types/shell.ts`: tipan las estructuras intercambiadas entre componentes, hooks y API.

Las consideraciones principales de diseño para esta capa fueron:

- **Consumo por rutas relativas**: toda comunicación con el backend se realiza mediante `/api/*`, y la resolución final se concentra en `next.config.ts`.
- **Renderizado integrado**: la aplicación principal se monta desde `page.tsx` y el shell administra el cambio de módulo sin abandonar la interfaz.
- **Visualización especializada**: el analizador muestra tablas y errores estructurados, el generador añade matriz de transiciones y renderizado SVG, y el gestor de pruebas incorpora ejecución y seguimiento de estados.
- **Manejo local de descargas**: la exportación de resultados se resuelve del lado del cliente para no introducir rutas auxiliares adicionales en la API.
- **Soporte de pruebas**: archivos como `vitest.config.ts`, `vitest.setup.ts` y las pruebas ubicadas junto a cada feature permiten validar hooks, componentes y flujos de interacción del frontend.

En conjunto, el frontend queda definido como una capa formada por una entrada de aplicación, un shell de navegación, features independientes por módulo, utilidades com-

partidas para API y descargas, y tipos explícitos que documentan la integración con el backend.

8. Integración entre Frontend y Backend

La comunicación entre capas se realiza mediante una API HTTP basada en FastAPI. El frontend consume rutas relativas y opera sobre contratos JSON tipados, lo que garantiza independencia entre la presentación y la lógica de compilación.

Método	Endpoint	Propósito
`GET`	`/api/health`	Verificar disponibilidad del servicio.
`GET`	`/api/dashboard/summary`	Obtener métricas generales para el dashboard.
`GET`	`/api/fixtures/catalog`	Obtener el catálogo de especificaciones y entradas disponibles.
`POST`	`/api/analysis/run`	Ejecutar análisis léxico sobre una entrada seleccionada o editada.
`POST`	`/api/generator/compile`	Compilar una especificación YALex y devolver autómatas, métricas y lexer generado.
`GET`	`/api/tests/catalog`	Obtener el catálogo de casos de prueba.
`POST`	`/api/tests/run`	Ejecutar un caso o un lote de casos y devolver resultados agregados.

Table 2: Endpoints principales de integración.

9. Módulos Funcionales

9.1 Dashboard

El dashboard resume información general del sistema: cantidad de especificaciones disponibles, cantidad de casos de prueba, distribución por complejidad y estado de los módulos operativos. Su función es ofrecer una vista consolidada del ecosistema completo del proyecto.

9.2 Lexical Analysis

Este módulo permite seleccionar una complejidad, cargar una entrada desde archivo o editarla manualmente y ejecutar el análisis léxico con el backend real. El resultado se presenta como una tabla de tokens, un panel de errores léxicos estructurados y estadísticas básicas de ejecución.

9.3 Lexical Generator

El generador permite editar o cargar una especificación YALex, compilarla y explorar sus resultados. Para cada compilación, la interfaz presenta:

- una regla activa seleccionable;
- la visualización del autómata correspondiente;
- la matriz de transiciones;
- la lista de tokens reconocidos;
- el código del lexer generado listo para descarga.

9.4 Test Case Manager

El administrador de casos de prueba consume un catálogo declarativo de entradas y especificaciones. Permite ejecutar casos individuales o lotes completos, registrar resultados de sesión y mostrar indicadores como porcentaje de éxito, ejecuciones acumuladas y casos pendientes de validación.

10. Generación de Lexer Independiente

Uno de los principios centrales del sistema es que el archivo Python generado sea completamente autosuficiente. El lexer exportado no depende del generador ni del paquete interno que lo produce. En su lugar, incorpora:

- la representación serializada de los autómatas;
- las estructuras básicas de tokens y errores;
- la función de escaneo;
- la lógica de prioridad y *longest match*;
- el entrypoint configurado por la especificación.

Esta propiedad garantiza portabilidad y hace posible utilizar el lexer generado como artefacto independiente dentro de otras herramientas o proyectos.

11. Casos de Prueba

La plataforma incluye casos de prueba organizados por complejidad funcional.

11.1 Complejidad Baja

La especificación de baja complejidad reconoce identificadores, enteros, operadores aritméticos y asignación. Este nivel valida el comportamiento básico del scanner y la omisión correcta de espacios en blanco.

```
variable + 5 * -11 = otraVariable
```

Listing 1: Ejemplo representativo de entrada para complejidad baja.

11.2 Complejidad Media

La complejidad media incorpora palabras reservadas, operadores de comparación y literales de cadena, ampliando el conjunto de patrones y el nivel de ambigüedad potencial en el proceso de matching.

```
if x1 >= 10 else while nombre == "abc"
```

Listing 2: Ejemplo representativo de entrada para complejidad media.

11.3 Complejidad Alta

La complejidad alta modela un subconjunto más expresivo de un lenguaje imperativo, con tipos, delimitadores, comparaciones compuestas, operadores lógicos y literales de punto flotante.

```
int x = 10;
float y = 20.5;
if (x <= y && x != 0) { return x; }
```

Listing 3: Ejemplo representativo de entrada para complejidad alta.

Cada nivel incluye entradas aceptadas y entradas con error, lo que permite validar tanto la identificación correcta de tokens como la detección precisa de fallos léxicos.

12. Estrategia de Pruebas y Validación

La calidad del sistema se valida mediante una estrategia de pruebas multinivel:

- **Pruebas unitarias del backend**: verifican tokenización, parsing, cálculo de funciones de posición, construcción de AFD, minimización, normalización YALex y serialización.
- **Pruebas de integración del backend**: validan el flujo completo entre parser, compilador, runtime y generación de código.

- **Pruebas de contrato**: verifican la forma exacta de las respuestas de la API y su consistencia para el frontend.
- **Pruebas de componentes e integración del frontend**: validan shell, análisis léxico, generación, ejecución de casos y descargas.
- **Pruebas end-to-end**: validan la experiencia completa de usuario desde la interfaz web hasta la ejecución real del backend.
- **Validación manual**: comprueba paridad visual, carga de archivos, descargas y ejecución de escenarios representativos.

Esta estrategia permite evaluar la corrección del sistema tanto desde la perspectiva algorítmica como desde la experiencia de uso final.

13. Resultados del Sistema

La plataforma ofrece un resultado integral sobre cuatro ejes principales:

1. **Compilación**: una especificación YALex se traduce a un conjunto de reglas compiladas con autómatas minimizados.
2. **Ejecución**: una entrada textual se procesa con política de *longest match*, devolviendo tokens, offsets, línea, columna y errores léxicos estructurados.
3. **Visualización**: el sistema muestra autómatas por regla, matrices de transición y tokens reconocidos desde una interfaz interactiva.
4. **Validación**: el conjunto de casos de prueba permite verificar el comportamiento del sistema para complejidades baja, media y alta.

La combinación de estos resultados convierte al proyecto en una plataforma didáctica y técnica capaz de mostrar tanto la teoría subyacente como su ejecución práctica.

14. Conclusiones

1. La plataforma demuestra que es posible integrar teoría de autómatas, compilación léxica y visualización interactiva en un solo sistema coherente.
2. La construcción directa de AFD a partir del árbol sintáctico ofrece una base formal sólida y evita dependencias en etapas intermedias innecesarias.
3. La minimización de estados mejora tanto la representación del autómata como la eficiencia del lexer generado.
4. La separación entre backend Python, API FastAPI y frontend web permite combinar exactitud algorítmica con una experiencia de usuario moderna.

5. La generación de un lexer autosuficiente refuerza el objetivo central del proyecto: producir analizadores léxicos reutilizables e independientes del entorno de compilación.
6. La estrategia de pruebas multinivel aporta una validación rigurosa del comportamiento del sistema desde la lógica interna hasta la interacción completa de usuario.

15. Referencias

[1] Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Addison-Wesley.

[2] Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Pearson.

[3] Leroy, X. et al. *The OCaml system: Documentation and user's manual.* Capítulo correspondiente a *ocamllex.*

[4] Valdez, C. (2026). *Consideraciones de YALex.* Universidad del Valle de Guatemala, CC3071.